

GitHub Copilot + Microsoft Foundry

Criando agentes de IA prontos para produção



Renato Romão

CTO · Belake.ai · Microsoft MVP

Renato Romão

Microsoft MVP

- 12+ years working as a Software Developer
- 6x Microsoft MVP
 - Copilot Studio
 - Microsoft Foundry
- 5x MCT
- 13,000+ students
- 11,000+ YouTube subscribers



Agenda

01

O cenário

Por que agentes de IA precisam de mais que código

02

GitHub Copilot

O copiloto que acelera o desenvolvimento

03

Microsoft Foundry

A plataforma para agentes em produção

04

A combinação

Arquitetura de referência e fluxo de trabalho

05

Demo & boas práticas

Do código ao agente em produção

O cenário

Construir um agente de IA não é só escrever código. É código + modelos + dados + governança.



Código

Lógica de negócio, integrações, APIs, testes. Aqui é onde o desenvolvedor passa o dia.



Modelos

GPT, Claude, modelos open-source, modelos especializados. Qual usar? Quanto custa? E latência?



Dados

Conhecimento corporativo, documentos, bases relacionais, vetoriais. Sem grounding, o agente alucina.



Governança

Identidade, autorização, observabilidade, compliance. Diferença entre POC e produção.

Agentes prontos para produção exigem que essas quatro camadas conversem.

GitHub Copilot

O copiloto que acelera o desenvolvimento de agentes



Geração de código

Funções, classes, integrações com SDKs do Foundry e Azure AI.



Plan Agent

Estrutura a lógica do agente antes de escrever uma linha de código.



Agent Mode

Lê o codebase, edita múltiplos arquivos, roda comandos e testa em loop.



MCP & extensões

Conecta Copilot a ferramentas externas — Foundry, GitHub, bases de dados.

Microsoft Foundry

A plataforma para construir, orquestrar e operar agentes em produção



Catálogo de modelos

GPT, Claude, Llama, Mistral, modelos open-source. Mesma API, troca por configuração.



Grounding corporativo

Conecta a SharePoint, Fabric, bases SQL, vetoriais. RAG gerenciado e seguro.



Agent Service

Orquestração de agentes, tools, memória, threads e estado — sem reinventar a roda.



Governança & evals

Identidade Entra ID, content safety, observabilidade, avaliação contínua.

A combinação

Cada um faz o que faz de melhor — e juntos cobrem o ciclo todo.



GitHub Copilot

Camada de DESENVOLVIMENTO

- ▶ Gera o código que chama o Foundry SDK
- ▶ Estrutura a lógica e o fluxo do agente
- ▶ Acelera testes, debug e refactoring
- ▶ Integra com MCP servers do Foundry
- ▶ Documenta e mantém o codebase vivo

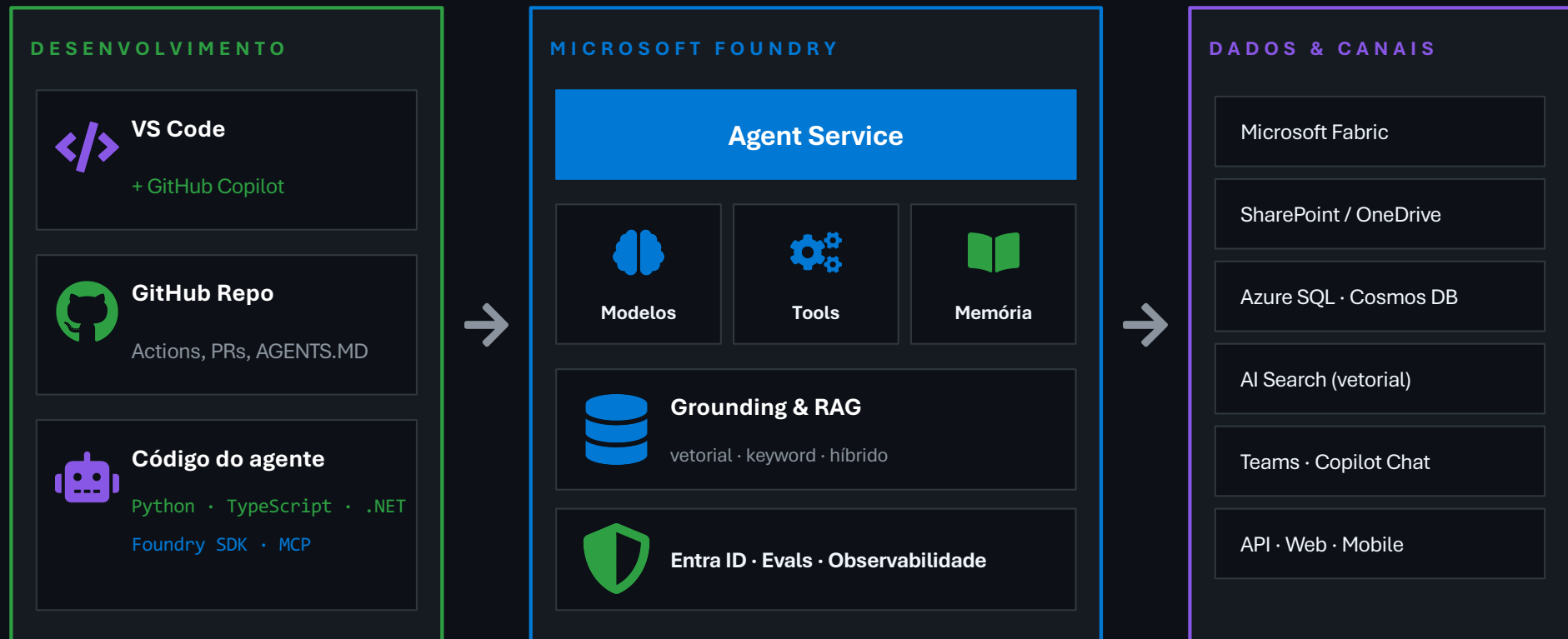


Microsoft Foundry

Camada de ORQUESTRAÇÃO

- ▶ Gerencia e troca modelos sem refazer código
- ▶ Faz grounding com dados corporativos
- ▶ Provê tools, memória e estado do agente
- ▶ Aplica governança, segurança e quotas
- ▶ Observa, avalia e versiona em produção

Arquitetura de referência



Do código ao agente em produção

Um fluxo onde Copilot e Foundry se entregam de mão dada.

1

PLAN

Plan Agent do Copilot estrutura o agente: tools, prompts, dados.

2

BUILD

Agent Mode gera o código que chama o Foundry SDK e configura tools.

3

GROUND

Foundry conecta a fontes corporativas — Fabric, SharePoint, SQL.

4

RUN

Agent Service orquestra modelo, ferramentas e estado em runtime.

5

OBSERVE

Evals contínuos, telemetria e ajuste fino — de volta pro código.

↻ *ciclo contínuo*

Na prática

Copilot pede o agente — Foundry entrega a infra.

PROMPT PARA O COPILOT

Crie um agente Foundry chamado "AssistentePolíticas" que:

- Use gpt-4o do catálogo Foundry
- Faça grounding em Azure AI Search (índice: politicas-rh-prod)
- Tenha uma tool que consulte a API interna de RH (OpenAPI: rh.json)
- Mantenha thread por usuário
- Aplique content safety padrão

Exporte como módulo Python e gere os testes unitários.



CÓDIGO GERADO · agent.py

```
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

client = AIProjectClient(
    endpoint=FOUNDRY_ENDPOINT,
    credential=DefaultAzureCredential(),
)

agent = client.agents.create_agent(
    model="gpt-4o",
    name="AssistentePolíticas",
    instructions=PROMPT_RH,
    tools=[
        {"type": "azure_ai_search",
         "index": "políticas-rh-prod"},
        {"type": "openapi",
         "spec": load("rh.json")},
    ],
)
```



Demo

VS Code + Copilot + Foundry — ao vivo

- 1 Plan Agent estrutura o agente
- 2 Agent Mode escreve o código
- 3 Foundry executa e responde no chat

Boas práticas

Lições do dia a dia construindo o Belake.



AGENTS.MD no repo

Documente convenções, modelos preferidos e tools — Copilot e o time leem o mesmo manual.



Comece pelo plan

Use Plan Agent antes de pedir código. Agente bem desenhado vale mais que 1.000 linhas.



Identidade primeiro

Entra ID e Managed Identity desde o dia 1. Não deixe credencial em código.



Grounding > prompt gigante

Não enfie tudo no system prompt. Use Foundry RAG e tools para dados frescos.



Evals como teste

Trate avaliação de agente como test suite. Roda em PR via GitHub Actions.



Modelo por tarefa

Modelo grande pra raciocínio, modelo pequeno pra extração. Foundry permite essa mistura.

O segredo não é escolher entre Copilot ou Foundry — é deixar cada um fazer o que faz melhor.

Links

- *Microsoft Foundry*
 - <https://ai.azure.com>
- *Skills para GitHub Copilot*
 - <https://skills.sh>
- *Applied skill – Criar um agente de IA*
 - <https://learn.microsoft.com/pt-br/credentials/applied-skills/create-an-ai-agent/>
- *Applied skill – Acelerar o desenvolvimento assistido por IA usando GitHub Copilot*
 - <https://learn.microsoft.com/pt-br/credentials/applied-skills/accelerate-app-development-by-using-github-copilot/>
- *Projetos de exemplo*
 - <https://github.com/renatoromao>

Obrigado!

Agora é a sua vez de combinar Copilot + Foundry.

Vamos continuar a conversa



linkedin.com/in/renatoromao



Renato Romão

Perguntas?

